

Unidad IV: Implementaciones dinámicas y eficiencia

Clase 6 – Listas dinámicas: cuando el arreglo no alcanza

Objetivo

Comprender la diferencia conceptual entre memoria estática y dinámica.

Tema

- Implementaciones dinámicas con nodos enlazados. Comparación con estáticas.

De las estructuras estáticas a las estructuras dinámicas

Hasta este momento, gran parte de las implementaciones trabajadas en la materia se apoyaron en **arreglos**. Un arreglo es una estructura muy útil porque permite guardar datos de manera simple y ordenada, pero tiene una restricción importante: su tamaño debe definirse al comienzo. Cuando se crea un arreglo en Java, se reserva memoria para una cantidad fija de posiciones. Por ejemplo, si se escribe `new int[100]`, se está indicando que la estructura podrá almacenar hasta 100 enteros. Ese límite queda fijado desde el inicio.

Este detalle, que al principio parece menor, se convierte en un problema cuando la cantidad de datos no puede predecirse con exactitud. Si la estructura necesita almacenar más elementos de los previstos, el arreglo no alcanza. Si, en cambio, se reserva un arreglo demasiado grande “por las dudas”, una parte de la memoria queda sin utilizar. En ambos casos aparece una ineficiencia: o bien la estructura queda chica, o bien se desperdicia espacio.

En una pila con capacidad fija, por ejemplo, si se intenta apilar el elemento 101 en una implementación creada con `new int[100]`, la estructura deja de ser suficiente para la necesidad real. En una cola, la situación es similar: si el flujo de elementos crece inesperadamente, el arreglo impone un techo. Esto se ve con más claridad todavía en sistemas del mundo real. En un sistema de e-commerce no siempre puede conocerse de antemano cuántos pedidos habrá durante una campaña. En un sistema de auditoría o monitoreo, la cantidad de eventos, alertas o transacciones puede crecer de forma impredecible. En un sistema de logs, la información registrada puede multiplicarse sin aviso previo. El problema, entonces, no es solo “guardar datos”, sino disponer de estructuras que acompañen el crecimiento real de la información.

Por eso, antes de estudiar listas dinámicas, es importante entender que estas no aparecen como un tema aislado, sino como la respuesta natural a una limitación concreta de las implementaciones estáticas.

Resumen de los comportamientos de los TDA trabajados

Antes de repasar las implementaciones, conviene recordar qué comportamiento define a cada TDA. Esta idea es central en la materia: el TDA describe qué hace la estructura, no cómo lo hace. La implementación es la que resuelve el cómo.

La **pila** responde al comportamiento LIFO, es decir, “Last In, First Out”. El último elemento que entra es el primero que sale. Sus operaciones principales son inicializar, apilar, desapilar, consultar el tope y verificar si está vacía.

La **cola** responde al comportamiento FIFO, “First In, First Out”. El primer elemento que entra es el primero que sale. Sus operaciones principales son inicializar, acolar, desacolar, consultar el primero y verificar si está vacía.

La **cola con prioridad** mantiene elementos asociados a una prioridad. Cuando se elimina o recupera un elemento, no necesariamente sale el primero que entró, sino el de mayor prioridad. Si dos elementos tienen la misma prioridad, sale primero el que ingresó antes.

El **conjunto** permite almacenar elementos sin repetición y sin orden. Como no hay orden interno, cuando se recupera un elemento se devuelve uno cualquiera. Sus operaciones principales son agregar, sacar, elegir, verificar pertenencia e inicializar.

El **diccionario simple** asocia a cada clave un único valor. Si se vuelve a agregar la misma clave con otro valor, el valor anterior se reemplaza. El **diccionario múltiple**, en cambio, asocia a cada clave un conjunto de valores, sin repetidos.

Este resumen debe quedar muy claro: antes de pensar en arreglos, nodos, índices o referencias, lo primero que hay que reconocer es el comportamiento lógico del TDA.

Repaso del modelo estático

Las implementaciones se construyen con arreglos. El apunte aclara que, cuando se implementa un TDA, se deben definir tres cuestiones: dónde se guarda la información, con qué criterio se guarda y recupera, y cómo se programan las operaciones. En el caso del modelo estático, ese “dónde” es siempre un arreglo de tamaño fijo.

1. Pila estática

Una de las implementaciones más importantes de repasar es PilaTF, es decir, pila con tope al final. En esta implementación se usa un arreglo para guardar los datos y una variable índice para indicar cuántos elementos hay en la pila.

```
public class PilaTF implements PilaTDA{
    int[] a;
    int indice;

    public void InicializarPila() {
        a = new int[100];
    }
}
```

```

    indice = 0;
}

public void Apilar(int x){
    a[indice] = x;
    indice++;
}

public void Desapilar() {
    indice--;
}

public boolean PilaVacía(){
    return (indice == 0);
}

public int Tope() {
    return a[indice - 1];
}
}

```

La lógica de esta implementación es simple. Cuando la pila está vacía, indice vale 0. Al apilar, el nuevo valor se guarda en la posición señalada por indice y luego se incrementa. Al desapilar, no hace falta borrar físicamente el valor: alcanza con decrementar indice, porque el tope lógico de la pila retrocede una posición. Consultar el tope consiste en mirar el elemento de la posición indice - 1. Esta implementación es muy eficiente porque no requiere corrimientos.

La otra implementación importante es PilaTI, donde el tope queda en la posición inicial del arreglo. En este caso, cada vez que se apila un elemento hay que correr todos los anteriores una posición a la derecha, y cuando se desapila ocurre un corrimiento hacia la izquierda.

```

public class PilaTI implements PilaTDA{
    int[] a;
    int indice;

    public void InicializarPila() {
        a = new int[100];
        indice = 0;
    }

    public void Apilar(int x){
        for (int i = indice - 1; i >= 0; i--)
            a[i + 1] = a[i];
        a[0] = x;
        indice++;
    }
}

```

```

}

public void Desapilar() {
    for (int i = 0; i < indice; i++)
        a[i] = a[i + 1];
    indice--;
}

public boolean PilaVacía(){
    return (indice == 0);
}

public int Tope() {
    return a[0];
}
}

```

Esta variante permite ver con claridad que dos implementaciones del mismo TDA pueden tener exactamente el mismo comportamiento externo, pero costos muy distintos. Ambas son pilas correctas; la diferencia está en cómo resuelven internamente el almacenamiento.

2. Cola estática

Para cola, el apunte muestra también dos variantes. En ColaPU, el último elemento ingresado se guarda en la posición 0, y para lograrlo es necesario correr hacia la derecha todos los elementos previos.

```

public class ColaPU implements ColaTDA{
    int[] a;
    int indice;

    public void InicializarCola() {
        a = new int[100];
        indice = 0;
    }

    public void Acolar(int x){
        for (int i = indice - 1; i >= 0; i--)
            a[i + 1] = a[i];
        a[0] = x;
        indice++;
    }

    public void Desacolar() {
        indice--;
    }
}

```

```
public boolean ColaVacia(){
    return (indice == 0);
}

public int Primero() {
    return a[indice - 1];
}
}
```

En esta estrategia, acolar es costoso porque obliga a desplazar elementos, pero desacolar es simple: basta con decrementar indice, ya que el primer elemento lógico está al final del sector ocupado.

En ColaPI, en cambio, el primer elemento ingresado queda en la posición 0. Esto simplifica Primero() y abarata Acolar(), pero hace más costoso Desacolar(), porque hay que correr todos los elementos hacia la izquierda.

```
public class ColaPI implements ColaTDA{
    int[] a;
    int indice;

    public void InicializarCola() {
        a = new int[100];
        indice = 0;
    }

    public void Acolar(int x){
        a[indice] = x;
        indice++;
    }

    public void Desacolar() {
        for (int i = 0; i < indice - 1; i++)
            a[i] = a[i + 1];
        indice--;
    }

    public boolean ColaVacia(){
        return (indice == 0);
    }

    public int Primero() {
        return a[0];
    }
}
```

Estas dos implementaciones son muy valiosas para estudiar porque muestran que el costo no depende solo de la estructura, sino de la decisión de diseño. En una, el costo fuerte se paga al insertar; en la otra, al eliminar.

3. Cola con prioridad estática

La cola con prioridad agrega un nivel más de complejidad. Una implementación presentada en el apunte es ColaPrioridadDA, basada en dos arreglos sincronizados: uno para los elementos y otro para sus prioridades.

```
public class ColaPrioridadDA implements ColaPrioridadTDA{
    int[] elementos;
    int[] prioridades;
    int indice;

    public void InicializarCola() {
        indice = 0;
        elementos = new int[100];
        prioridades = new int[100];
    }

    public void AcolarPrioridad(int x, int prioridad){
        int j = indice;
        for (; j > 0 && prioridades[j - 1] >= prioridad; j--) {
            elementos[j] = elementos[j - 1];
            prioridades[j] = prioridades[j - 1];
        }
        elementos[j] = x;
        prioridades[j] = prioridad;
        indice++;
    }

    public void Desacolar() {
        indice--;
    }

    public int Primero() {
        return elementos[indice - 1];
    }

    public boolean ColaVacia(){
        return (indice == 0);
    }

    public int Prioridad() {
        return prioridades[indice - 1];
    }
}
```

```
}  
}
```

Aquí la operación central es `AcolarPrioridad()`. Como la estructura debe mantenerse ordenada por prioridad, el nuevo elemento no siempre se agrega al final: muchas veces debe insertarse en el medio, corriendo elementos para conservar el orden correcto. Por eso el costo fuerte se concentra en la inserción.

4. Conjunto estático

La implementación estática del conjunto aprovecha un rasgo esencial de este TDA: no hay orden.

```
public class ConjuntoTA implements ConjuntoTDA{  
    int[] a;  
    int cant;  
  
    public void InicializarConjunto() {  
        a = new int[100];  
        cant = 0;  
    }  
  
    public void Agregar(int x) {  
        if (!this.Pertenece(x)) {  
            a[cant] = x;  
            cant++;  
        }  
    }  
  
    public boolean ConjuntoVacio() {  
        return cant == 0;  
    }  
  
    public int Elegir() {  
        return a[cant - 1];  
    }  
  
    public boolean Pertenece(int x) {  
        int i = 0;  
        while (i < cant && a[i] != x)  
            i++;  
        return (i < cant);  
    }  
  
    public void Sacar(int x) {  
        int i = 0;  
        while (i < cant && a[i] != x)
```

```

        i++;
    if (i < cant) {
        a[i] = a[cant - 1];
        cant--;
    }
}
}

```

La clave didáctica de esta implementación está en Sacar(). Como el conjunto no conserva orden, no es necesario correr elementos al eliminar uno. Se puede reemplazar el elemento borrado por el último del arreglo y decrementar la cantidad. Este detalle es muy importante porque muestra cómo el comportamiento lógico del TDA condiciona las decisiones de implementación.

5. Diccionario estático

En el diccionario simple, cada elemento del arreglo representa una asociación clave-valor. El apunte define una clase interna Elemento y luego trabaja con un arreglo de esos elementos.

```

public class DicSimpleA implements DiccionarioSimpleTDA {
    class Elemento{
        int clave;
        int valor;
    }

    Elemento[] elementos;
    int cant;

    public void InicializarDiccionario() {
        cant = 0;
        elementos = new Elemento[100];
    }

    public void Agregar(int clave, int valor){
        int pos = Clave2Indice(clave);
        if (pos == -1) {
            pos = cant;
            elementos[pos] = new Elemento();
            elementos[pos].clave = clave;
            cant++;
        }
        elementos[pos].valor = valor;
    }

    private int Clave2Indice(int clave){
        int i = cant - 1;
    }
}

```

```

while(i >= 0 && elementos[i].clave != clave)
    i--;
return i;
}

public void Eliminar(int clave) {
    int pos = Clave2Indice(clave);
    if (pos != -1) {
        elementos[pos] = elementos[cant - 1];
        cant--;
    }
}

public int Recuperar(int clave) {
    int pos = Clave2Indice(clave);
    return elementos[pos].valor;
}
}

```

Otra vez aparece una idea conocida: como las claves no necesitan conservar un orden particular, al eliminar puede colocarse en su lugar el último elemento ocupado. Esto evita corrimientos y simplifica la operación. El costo principal, sin embargo, sigue estando en la búsqueda secuencial de la clave.

Costos de las implementaciones estáticas

El apunte dedica una sección específica al análisis de costos y aclara que, en esta materia, se utilizará una clasificación simplificada de complejidad temporal. Las categorías son costo constante, costo lineal y costo polinómico. El costo constante significa que la cantidad de operaciones no depende de la cantidad de elementos de entrada. El costo lineal significa que el trabajo crece en proporción a la cantidad de elementos. El costo polinómico agrupa costos superiores a lineales, por ejemplo cuadráticos o cúbicos.

Si se lo compara con la notación Big-O más habitual, puede pensarse así: costo constante equivale aproximadamente a $O(1)$; costo lineal equivale a $O(n)$; costo polinómico agrupa casos como $O(n^2)$ u $O(n^3)$. En cursos más avanzados suele distinguirse también $O(\log n)$, pero en el esquema de esta materia no aparece como categoría separada. Si alguna vez surge en otro contexto, debe entenderse que es mejor que $O(n)$, aunque aquí no se lo clasifique aparte.

Los costos más representativos del repaso estático son los siguientes, según la tabla del apunte. En PilaTF, todas las operaciones son de costo constante. En PilaTI, apilar y desapilar son lineales. En ColaPU, acolar es lineal y desacolar es constante. En ColaPI, ocurre lo contrario: acolar es constante y desacolar lineal. En las colas con prioridad estáticas, la inserción es lineal y el resto de las operaciones son constantes. En

ConjuntoTA, agregar, sacar y pertenece son lineales, mientras que elegir es constante. En los diccionarios, salvo la inicialización, las operaciones principales son lineales.

Esto permite sacar una conclusión muy importante: el uso de arreglos no implica necesariamente mal desempeño, pero sí suele traer dos dificultades. La primera es el tamaño fijo. La segunda es que muchas operaciones requieren recorridos o corrimientos, y eso hace que el costo crezca con la cantidad de elementos.

Concepto de implementación dinámica

Después de entender bien las limitaciones del modelo estático, puede introducirse el modelo dinámico. Las implementaciones dinámicas se basan en memoria dinámica. A diferencia de los arreglos, donde la memoria se reserva por completo al momento de crear la estructura, en una implementación dinámica la memoria se va reservando a medida que se necesita. Esto significa que la estructura puede crecer gradualmente, sin necesidad de conocer por adelantado cuántos elementos contendrá.

En Java, además, la memoria que deja de utilizarse se libera de manera automática mediante el mecanismo de Garbage Collection. Esto quiere decir que, cuando un nodo deja de estar referenciado por la estructura, el programador no necesita liberarlo manualmente como ocurriría en otros lenguajes.

La base de estas implementaciones es el **nodo**. Un nodo es una estructura que guarda un valor y una referencia a otro nodo. El apunte lo presenta así:

```
class Nodo {
    int info;
    Nodo sig;
}
```

El campo info guarda el dato. El campo sig guarda una referencia al siguiente nodo de la cadena. Si no existe un siguiente nodo, esa referencia vale null. Así, una estructura dinámica lineal puede imaginarse como una secuencia de nodos enlazados unos con otros.

Este cambio es muy profundo. En el modelo estático, los elementos estaban uno al lado del otro en posiciones consecutivas del arreglo. En el modelo dinámico, los elementos ya no se ubican necesariamente juntos en memoria: se conectan mediante referencias. Por eso, en vez de hablar de índices o posiciones, ahora se habla de nodos, enlaces, primero, último, siguiente o referencias.

De lo estático a lo dinámico: el nuevo clic conceptual

El paso del arreglo a la lista dinámica implica cambiar la forma de pensar las estructuras. En el arreglo, se trabaja con una zona continua de memoria y con posiciones numeradas. En la lista dinámica, en cambio, lo importante es la relación

entre nodos. Cada nodo “sabe” dónde está el siguiente. La estructura deja de apoyarse en un tamaño fijo y pasa a organizarse como una cadena de objetos enlazados.

Este paso no modifica el comportamiento del TDA. Una pila sigue siendo una pila. Una cola sigue siendo una cola. Un conjunto sigue siendo un conjunto. Lo que cambia es la manera de implementarlos. Esto es fundamental: la especificación del TDA permanece igual; lo que cambia es la estructura interna utilizada para lograr ese comportamiento.

Aspecto	Modelo estático	Modelo dinámico
Estructura base	Arreglo	Nodos enlazados
Reserva de memoria	Completa al inicio	Gradual, a medida que se necesita
Tamaño	Fijo	Variable
Crecimiento	Limitado por la capacidad elegida	Flexible, según memoria disponible
Organización interna	Posiciones e índices	Referencias y enlaces
Corrimientos	Frecuentes en varias operaciones	Muchas veces se evitan
Facilidad conceptual inicial	Más simple de visualizar al comienzo	Requiere comprender referencias
Problema principal	Puede sobrar o faltar memoria	Mayor complejidad conceptual
Relación con el TDA	Implementa el mismo comportamiento	Implementa el mismo comportamiento

La conclusión de esta tabla es clara. El modelo estático fue muy útil para aprender a implementar TDAs y analizar costos. Pero también mostró un límite fuerte: la capacidad fija y la necesidad de corrimientos en muchas operaciones. El modelo dinámico aparece entonces como una evolución natural. Permite que la estructura crezca de acuerdo con la necesidad real, y en varios casos mejora costos al evitar desplazamientos de elementos.

Encadenamiento: la idea de lista enlazada

Cuando trabajábamos con arreglos, los elementos estaban organizados por posición. Cada dato ocupaba un lugar numerado dentro de una estructura continua: posición 0, 1, 2, 3, y así sucesivamente. En una lista enlazada la lógica cambia por completo. Los elementos ya no están relacionados por una posición fija dentro de un bloque de memoria, sino por una referencia que conecta un nodo con el siguiente.

Si representamos una lista como:

$[5 \mid *] \rightarrow [8 \mid *] \rightarrow [3 \mid \text{null}]$

lo que estamos viendo no es simplemente una secuencia de valores, sino una cadena de nodos. Cada nodo tiene dos partes. La primera guarda la información, en este caso 5, 8 y 3. La segunda guarda una referencia al nodo siguiente. Esa referencia es lo que “encadena” la estructura. El apunte de la cátedra explica justamente que la base de una estructura dinámica es el nodo, y que en cada nodo se define un espacio para almacenar el valor y otro para guardar una referencia a otro nodo. Además, aclara que cuando un nodo no apunta a ningún otro, esa referencia toma el valor especial null.

A partir de esta idea aparecen tres conceptos fundamentales. El primero es **primero** o **head**, que es la referencia al nodo inicial de la lista. Esta referencia es clave porque constituye la puerta de entrada a toda la estructura. Si se pierde la referencia al primer nodo, se pierde el acceso al resto de la cadena. El segundo concepto es **sig**, abreviatura de “siguiente”, que es la referencia que permite pasar de un nodo al próximo. El tercero es **null**, que no representa un dato, sino la ausencia de un siguiente nodo. Por eso se usa para marcar el final de la lista.

Este punto es central porque muestra el cambio de mentalidad que exige la estructura dinámica. En un arreglo uno piensa “quiero ir a la posición 7”. En una lista enlazada uno piensa “empiezo en el primero y sigo avanzando nodo a nodo hasta llegar al que necesito”. Es decir, la estructura deja de basarse en posiciones y pasa a basarse en enlaces.

Operaciones básicas sobre listas

Antes de aplicar esta idea a pila, cola, conjunto o diccionario, conviene entender las operaciones más elementales sobre una lista enlazada simple. Las dos más didácticas son insertar al inicio y eliminar al inicio, porque permiten ver con claridad cómo se manipulan las referencias sin necesidad de recorrer toda la estructura.

Cuando se inserta un nodo al inicio, lo primero que se hace es crear el nuevo nodo y cargar en él el valor que se quiere guardar. Pero ese nodo nuevo todavía está aislado. Para enlazarlo con la lista existente, su campo sig debe apuntar al nodo que hasta ese momento era el primero. Recién después se actualiza la referencia primero para que pase a señalar al nuevo nodo. Conceptualmente, esto significa que el nuevo nodo se convierte en la nueva entrada a la estructura, y el resto de la lista queda “colgado” de él.

Lo importante aquí es entender por qué esta operación es tan eficiente. No hace falta mover elementos, no hay corrimientos, no hay que buscar una posición libre dentro de un arreglo. Solamente se crean y actualizan referencias. Por eso esta inserción se considera de costo constante, es decir, $O(1)$. El tiempo que lleva no depende de cuántos nodos haya en la lista. Si la lista tiene 3 nodos o 3.000, insertar al inicio requiere siempre la misma cantidad básica de pasos. Esta es una de las razones por las que esta operación resulta tan importante en estructuras dinámicas. Además, desde el punto de vista didáctico, constituye la base directa para entender la pila dinámica.

Eliminar al inicio es todavía más revelador. Conceptualmente, eliminar el primer nodo no implica “destruirlo” manualmente como uno podría imaginar. Lo que se hace es cambiar la referencia primero para que deje de apuntar al nodo actual y pase a apuntar al siguiente. Si antes primero apuntaba al nodo que contenía 5, luego de la operación pasará a apuntar al nodo que contenía 8. El antiguo primer nodo queda desconectado de la estructura. A partir de ese momento, la lista comienza en el que era el segundo nodo.

Este ejemplo también es muy útil para explicar una particularidad de Java. En el apunte se señala que la memoria que ya no se necesita queda automáticamente disponible gracias al mecanismo de recolección de basura o Garbage Collection. Esto significa que, cuando un nodo deja de estar referenciado por la estructura y no existe ninguna otra variable que apunte a él, Java podrá liberar esa memoria de manera automática. Desde el punto de vista del alumno, esto permite concentrarse en la lógica de la estructura sin tener que administrar manualmente la memoria.

Lo esencial de estas dos operaciones es que muestran una diferencia profunda con el modelo estático. En el arreglo, insertar o eliminar muchas veces implicaba desplazar varios elementos. En la lista enlazada, al menos en el extremo inicial, lo que cambia son las referencias. La estructura no se reorganiza moviendo datos, sino reencadenando nodos.

Relación directa con los TDA ya vistos

Una vez comprendida la lógica de la lista enlazada simple, resulta mucho más fácil relacionarla con los TDA que ya se trabajaron anteriormente. La idea fundamental es que la estructura dinámica no cambia el comportamiento del TDA; lo que cambia es la forma interna de implementarlo. En otras palabras, la pila sigue siendo LIFO, la cola sigue siendo FIFO, el conjunto sigue sin orden ni repetidos, y el diccionario sigue funcionando con claves. Lo nuevo está en cómo se materializa internamente cada una de esas conductas.

En el caso de la **pila**, la relación es casi inmediata. La operación de apilar coincide conceptualmente con insertar al inicio de la lista, y la operación de desapilar coincide con eliminar al inicio. Esto hace que la lista enlazada sea especialmente natural para implementar una pila dinámica. Además, permite conservar costos constantes en las operaciones centrales, sin recurrir a corrimientos como ocurría en algunas versiones estáticas.

En la **cola**, en cambio, la situación exige una idea adicional. Si solo se mantuviera la referencia al primero, insertar al final obligaría a recorrer toda la lista. Por eso la implementación dinámica de cola suele conservar dos referencias: una al primero y otra al último. Así, se puede agregar por un extremo y eliminar por el otro respetando el comportamiento FIFO. Comparada con las colas estáticas, la ventaja principal es que desaparecen los corrimientos de elementos. El costo pasa a concentrarse en la manipulación de enlaces.

En el **conjunto**, como no existe orden, la lista enlazada se usa más bien como una secuencia de nodos a recorrer. Insertar puede ser sencillo, pero para verificar pertenencia o eliminar un elemento determinado sigue siendo necesario buscarlo. Por eso la implementación dinámica no elimina todos los recorridos, aunque sí evita la rigidez del tamaño fijo del arreglo.

En los **diccionarios**, la lógica es parecida. La lista enlazada permite guardar asociaciones de clave y valor sin un límite fijo predeterminado. Sin embargo, para ubicar una clave específica todavía suele ser necesario recorrer la estructura. Por eso, al pasar de estático a dinámico, no siempre mejoran todos los costos; muchas veces lo que mejora principalmente es la flexibilidad del almacenamiento.

La **cola con prioridad** agrega otro matiz. En una versión dinámica, la lista suele mantenerse ordenada por prioridad. Esto hace que la eliminación del elemento prioritario sea sencilla, porque se encuentra en una posición conocida de la cadena, pero insertar puede requerir recorrido hasta encontrar el lugar correcto. Nuevamente aparece una idea ya conocida por los alumnos: según cómo se organice la estructura internamente, el costo “se paga” en operaciones distintas.

Visto en conjunto, el paso a listas dinámicas no borra lo aprendido con las implementaciones estáticas. Al contrario: lo profundiza. Permite volver a pensar los mismos TDA, pero ahora desde otro soporte físico y otra lógica de organización.

Ventajas y desventajas de las estructuras dinámicas

Las estructuras dinámicas presentan ventajas muy importantes frente a las implementaciones estáticas. La primera es que no tienen un tamaño fijo predeterminado. Esto significa que la estructura puede crecer a medida que se agregan nodos, sin necesidad de reservar desde el comienzo un bloque grande de memoria “por las dudas”. Esta flexibilidad es especialmente valiosa cuando la cantidad de datos no puede conocerse con anticipación. El apunte subraya precisamente esta idea al explicar que, en las estructuras dinámicas, la memoria se va reservando a medida que se necesita.

La segunda ventaja es que ciertas inserciones se vuelven muy eficientes. Insertar al inicio, por ejemplo, requiere solo unas pocas asignaciones de referencias y no depende de la cantidad total de nodos. Esto permite operaciones de costo constante que en algunas implementaciones estáticas podían requerir recorridos o corrimientos.

La tercera ventaja es que desaparecen los corrimientos de elementos. En un arreglo, cuando se quiere insertar o borrar en una posición determinada, muchas veces hay que mover varios valores. En una lista enlazada, en cambio, la estructura se reorganiza modificando referencias. Esto no significa que todo sea siempre más rápido, porque algunas búsquedas siguen necesitando recorrido, pero sí implica una forma de trabajo mucho más flexible.

Sin embargo, las estructuras dinámicas también tienen desventajas. La primera es el **uso extra de memoria**. Cada nodo no solo guarda un dato, sino también una referencia al siguiente. Es decir, parte del espacio total no se dedica al contenido en sí, sino a la información necesaria para enlazar la estructura.

La segunda desventaja es la **mayor complejidad conceptual**. Para un alumno, un arreglo suele ser más intuitivo al comienzo porque se basa en posiciones numeradas. En cambio, una lista enlazada exige pensar en objetos, referencias, nodos desconectados, cambios de enlaces y recorridos secuenciales. El desafío ya no es solo almacenar datos, sino comprender cómo fluye la referencia entre nodos.

La tercera desventaja es la ausencia de **acceso directo por índice**. En un arreglo, si se quiere consultar la posición 7, se accede directamente a ella. En una lista enlazada, no existe esa posibilidad. Para llegar al séptimo nodo hay que comenzar desde el primero y avanzar uno por uno. Esto significa que algunas operaciones que en un arreglo son inmediatas, en una lista enlazada requieren recorrido.

En síntesis, la implementación dinámica no debe pensarse como “mejor en todo” respecto de la estática. Lo correcto es entender que resuelve muy bien ciertos problemas, especialmente la flexibilidad de tamaño y la eliminación de corrimientos, pero a cambio introduce más complejidad conceptual, mayor uso estructural de memoria y pérdida del acceso directo por índice. Esa comparación es la que realmente permite evaluar cuándo una estructura conviene más que otra.

La idea clave que debe quedar fijada es esta: las listas enlazadas cambian la forma de organizar la información. En lugar de depender de posiciones fijas dentro de un arreglo, dependen de nodos conectados por referencias. Ese cambio es el que abre la puerta a las implementaciones dinámicas de los TDA que ya venían estudiando.

Implementación con estructuras dinámicas

En esta unidad vamos a implementar los TDA vistos en unidades anteriores utilizando ahora estructuras dinámicas. Una vez implementados, vamos a comparar los costos de estas implementaciones contra las anteriores. Al final de la unidad vamos a resolver problemas combinando estos TDA y más de una implementación de un mismo TDA.

4.1 Implementaciones dinámicas

Las implementaciones dinámicas se basan en el concepto de utilizar memoria dinámica, es decir que, a diferencia de los arreglos, para los cuales definimos y reservamos la cantidad de memoria al momento de su creación, en las estructuras dinámicas, vamos reservando memoria a medida que se necesita. Asimismo la memoria que ya no se necesita se devuelve al sistema. Por las características de Java, la idea de devolver memoria es implícita, es decir, la memoria que no se utiliza, automáticamente queda disponible. Este mecanismo se denomina Garbage

Collection.

La base de una estructura dinámica es lo que llamamos célula o nodo. La idea entonces es especificar un nodo que tenga la capacidad de almacenar un valor. Luego, se van a encadenar tantos nodos como sean necesarios.

Cabe recordar aquí que esta implementación no afectará en absoluto el comportamiento del TDA, es decir que todo aquel que haya usado el TDA con una implementación estática, lo podrá seguir usando de la misma forma, con una implementación dinámica sin modificar nada.

Volviendo a los nodos y la forma de encadenarlos, en cada nodo se definirá el espacio para almacenar el valor y para guardar una referencia a otro nodo. Así con un nodo, tenemos el valor y la referencia a otro nodo. Esto da lugar a una cadena de nodos potencialmente infinita. Un nodo que no apunta a ningún otro nodo, tendrá un valor especial en esa posición: null. La estructura del nodo se puede ver en la figura 4.1.

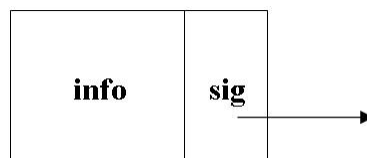


Figura 4.1: Nodo de la estructura dinámica lineal

Así, dos nodos encadenados quedan como en la figura 4.2.

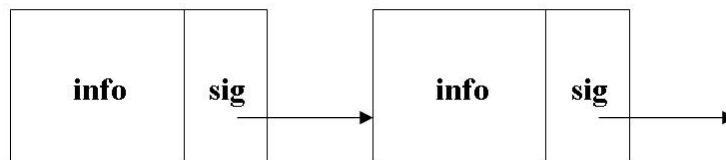


Figura 4.2: Dos nodos encadenados de la estructura dinámica lineal

Con esto, se puede generalizar a N nodos.

Entonces, en Java, la estructura del nodo sería:

```
class Nodo{
    int info;
    Nodo sig;
}
```

4.1.1 Pila

Vamos ahora a implementar el PilaTDA con una estructura dinámica. Para ello, recordemos la especificación del PilaTDA:

```
public interface PilaTDA {
    void InicializarPila();
    void Apilar(int x);
    void Desapilar();
    boolean PilaVacía();
    int Tope();
}
```

La estrategia entonces será mantener una cadena de nodos, manteniendo un puntero al tope de la pila, este apuntará al siguiente, y así sucesivamente. Si la lista está vacía, será un puntero null. Para agregar un nodo, se lo crea, y se agrega al comienzo de la cadena. Para eliminarlo, simplemente se elimina el primer nodo, dejando el puntero apuntando al segundo si es que existe.

```
public class PilaLD implements PilaTDA{

    Nodo primero;

    public void InicializarPila(){
        primero = null;
    }

    public void Apilar(int x){
        Nodo aux = new Nodo();
        aux.info = x;
        aux.sig = primero;
        primero = aux;
    }

    public void Desapilar(){
        primero = primero.sig;
    }

    public boolean PilaVacía(){
        return (primero == null);
    }

    public int Tope(){
        return primero.info;
    }
}
```

4.1.2 Cola

Recordemos la especificación del ColaTDA:

```
public interface ColaTDA {
    void InicializarCola();
    void Acolar(int x);
    void Desacolar();
    boolean ColaVacía();
    int Primero();
}
```

Con la misma estrategia que para la pila, ahora vamos implementar la cola, esta vez, manteniendo un puntero al primer elemento y uno al último ya que por el comportamiento de este TDA, se deberá agregar por un extremo y eliminar por el otro. El orden en que estarán los elementos será el primero apunta al segundo, el segundo al tercero, así siguiendo, llegando al último elemento de la cadena que será el último elemento agregado a la Cola.

```
public class ColaLD implements ColaTDA{

    //Primer elemento en la cola
    Nodo primero;
```

```
//Último elemento en la cola, es decir, el último agregado
Nodo ultimo;

public void InicializarCola(){
    primero = null;
    ultimo = null;
}

public void Acolar(int x){

    Nodo aux = new Nodo();
    aux.info = x;
    aux.sig = null;
    //Si la cola no está vacía
    if(ultimo != null)
        ultimo.sig = aux;
    ultimo = aux;

    // Si la cola estaba vacía
    if(primero == null)
        primero = ultimo;
}

public void Desacolar(){
    primero = primero.sig;

    // Si la cola queda vacía
    if(primero == null)
        ultimo = null;
}

public boolean ColaVacía(){
    return (ultimo == null);
}

public int Primero(){
    return primero.info;
}
}
```

4.1.3 Cola con Prioridad

Recordemos la especificación del TDA ColaPrioridad:

```
public interface ColaPrioridadTDA {
    void InicializarCola();

    void AcolarPrioridad(int x, int prioridad);

    void Desacolar();

    int Primero();

    boolean ColaVacía();

    int Prioridad();
}
```

Aquí la estrategia será modificar el nodo, para que además del valor tenga la prioridad incluida:

```
class NodoPrioridad{
    int info;
    int prioridad;
    NodoPrioridad sig;
}
```

Y luego implementar el TDA manteniendo la cadena de nodos ordenada primero por prioridad y luego por orden de entrada, manteniendo un puntero al elemento con mayor prioridad, que es el primero que se deberá eliminar.

```
public class ColaPrioridadLD implements ColaPrioridadTDA {

    NodoPrioridad mayorPrioridad;

    public void InicializarCola(){
        mayorPrioridad = null;
    }

    public void AcolarPrioridad(int x, int prioridad){

        // Creo el nuevo nodo que voy a acolar
        NodoPrioridad nuevo = new NodoPrioridad();
        nuevo.info = x;
        nuevo.prioridad = prioridad;

        // Si la cola está vacía o bien es más prioritario que
        // el primero hay que agregarlo al principio
        if(mayorPrioridad == null ||
            prioridad > mayorPrioridad.prioridad) {
            nuevo.sig = mayorPrioridad;
            mayorPrioridad = nuevo;
        }
        else {
            //Sabemos que mayorPrioridad no es null
            NodoPrioridad aux = mayorPrioridad;

            while(aux.sig!=null && aux.sig.prioridad>=prioridad){
                aux = aux.sig;
            }

            nuevo.sig = aux.sig;
            aux.sig = nuevo;
        }
    }
}
```

```

public void Desacolar(){
    mayorPrioridad = mayorPrioridad.sig;
}

public int Primero(){
    return mayorPrioridad.info;
}

public boolean ColaVacia(){
    return (mayorPrioridad == null);
}

public int Prioridad(){
    return mayorPrioridad.prioridad;
}
}

```

4.1.4 Conjuntos

Para implementar el conjunto, a diferencia de las otras estructuras, no hay orden, por lo tanto, la estrategia es simplemente mantener la cadena de nodos y siempre se deberá recorrerla para buscar un elemento o eliminarlo. Aquí presentamos una implementación, existen otras que mantienen la cadena ordenada. Con esto se mejoran algunos costos y se empeoran otros.

Recordemos entonces la especificación del ConjuntoTDA:

```

public interface ConjuntoTDA {
    void InicializarConjunto();
    boolean ConjuntoVacio();
    void Agregar(int x);
    int Elegir();
    void Sacar(int x);
    boolean Pertenece(int x);
}

```

Y la implementación propuesta es la siguiente:

```

public class ConjuntoLD implements ConjuntoTDA {

    Nodo c;

    public void InicializarConjunto() {
        c = null;
    }

    public boolean ConjuntoVacio() {
        return (c == null);
    }
}

```

```

public void Agregar(int x) {
    /* Verifica que x no este en el conjunto */
    if (!this.Pertenece(x)){
        Nodo aux = new Nodo();
        aux.info = x;
        aux.sig = c;
        c = aux;
    }
}

public int Elegir() {
    return c.info;
}

public void Sacar(int x) {
    if(c!=null){
        // si es el primer elemento de la lista
        if (c.info == x) {
            c = c.sig;
        }else{
            Nodo aux = c;
            while (aux.sig!=null && aux.sig.info!=x)
                aux = aux.sig;
            if (aux.sig != null)
                aux.sig = aux.sig.sig;
        }
    }
}

public boolean Pertenece(int x) {
    Nodo aux = c;
    while ((aux != null) && (aux.info != x)){
        aux = aux.sig;
    }
    return (aux != null);
}
}

```

4.1.5 Diccionarios

4.1.5.1 Diccionario Simple

La implementación propuesta es la siguiente:

```
public class DicSimpleL implements DiccionarioSimpleTDA {

    class NodoClave{
        int clave;
        int valor;
        NodoClave sigClave;
    }

    NodoClave origen;

    public void InicializarDiccionario(){
        origen = null;
    }

    public void Agregar(int clave, int valor){
        NodoClave nc = Clave2NodoClave( clave);
        if (nc==null) {
            nc = new NodoClave();
            nc.clave = clave;
            nc.sigClave = origen;
            origen = nc;
        }
        nc.valor = valor;
    }

    private NodoClave Clave2NodoClave(int clave){
        NodoClave aux = origen;
        while (aux!=null && aux.clave!=clave){
            aux = aux.sigClave;
        }
        return aux;
    }

    public void Eliminar(int clave) {
        if(origen!=null) {
            if(origen.clave == clave) {
                origen = origen.sigClave;
            }
            else {
                NodoClave aux = origen;
                while (aux.sigClave != null && aux.sigClave.clave
                    !=clave){
                    aux = aux.sigClave;
                }
                if(aux.sigClave!=null) {
                    aux.sigClave= aux.sigClave.sigClave;
                }
            }
        }
    }
}
```

```

public int Recuperar(int clave){
    NodoClave n = Clave2NodoClave(clave);
    return n.valor;
}

public ConjuntoTDA Claves(){
    ConjuntoTDA c = new ConjuntoLD();
    c.InicializarConjunto();

    NodoClave aux = origen;
    while (aux != null){
        c.Agregar(aux.clave);
        aux = aux.sigClave;
    }
    return c;
}
}

```

4.1.5.2 Diccionario Múltiple

La implementación propuesta es la siguiente:

```

public class DicMultipleL implements DiccionarioMultipleTDA {

    class NodoClave{
        int clave;
        NodoValor valores;
        NodoClave sigClave;
    }

    class NodoValor{
        int valor;
        NodoValor sigValor;
    }

    NodoClave origen;

    public void InicializarDiccionario(){
        origen = null;
    }

    public void Agregar(int clave, int valor){
        NodoClave nc = Clave2NodoClave( clave);
        if (nc==null) {
            nc= new NodoClave(); nc.clave=clave;

            nc.sigClave= origen;

            origen = nc;
        }

        NodoValor aux = nc.valores;

        while (aux!= null && aux.valor!=valor) {
            aux= aux.sigValor;}

        if (aux== null) {

```

```

        NodoValor nv = new NodoValor();

        nv.valor = valor;

        nv.sigValor = nc.valores;

        nc.valores = nv;

    }

}

private NodoClave Clave2NodoClave( int clave){

    NodoClave aux = origen;

    while (aux!= null && aux.clave!=clave){

        aux = aux.sigClave;

    }

    return aux;

}

public void EliminarValor( int clave , int valor){

    if (origen!= null) {

        if (origen.clave == clave) {

            EliminarValorEnNodo(origen , valor);

            if (origen.valores== null) {

                origen = origen.sigClave;

            }

        }

        else {

            NodoClave aux = origen;

            while (aux.sigClave != null && aux.sigClave.clave!=clave){

                aux = aux.sigClave;

            }

            if (aux.sigClave!= null) {

                EliminarValorEnNodo(aux.sigClave , valor);

                if (aux.sigClave.valores== null) {

                    aux.sigClave= aux.sigClave.sigClave;

                }

            }

        }

    }

}

```

```

    }

    }

}

private void EliminarValorEnNodo(NodoClave nodo, int valor) { if (nodo.valores!=null)
{
    if (nodo.valores.valor == valor) { nodo.valores = nodo.valores.sigValor;
        }
        else {
            else {
                NodoClave aux = origen;
                while (aux.sigClave != null && aux.sigClave.clave
                    !=clave){ aux = aux.sigClave;
                } if (aux.sigClave!=null) { aux.sigClave= aux.sigClave.sigClave; }
            }
        }
    }
}

public ConjuntoTDA Recuperar(int clave){
    NodoClave n = Clave2NodoClave(clave);

    ConjuntoTDA c = new ConjuntoLD(); c.InicializarConjunto();
    if (n!=null) {
        NodoValor aux = n.valores; while (aux != null){
            c.Agregar(aux.valor);
            aux = aux.sigValor;
        }
    }
    return c;
}

public ConjuntoTDA Claves(){
    ConjuntoTDA c = new ConjuntoLD(); c.InicializarConjunto();

    NodoClave aux = origen;
    while (aux != null){
        c.Agregar(aux.clave);
        aux = aux.sigClave;
    }
    return c;
}
}

```

•